

Kishore S

CH.SC.U4CSE24222

Week – 5

Design and Analysis of Algorithm(23CSE211)

AVL and Red - Black Tree

1. Write a c program for AVL Tree

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key, height;
    struct Node *left, *right;
};

int height(struct Node *n) { return n ? n->height : 0; }
int max(int a, int b) { return (a > b) ? a : b; }

struct Node* newNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}
```

```

struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}

```

```

int getBalance(struct Node* n) { return n ? height(n->left) - height(n->right) : 0; }

```

```

struct Node* insert(struct Node* node, int key) {
    if (!node) return newNode(key);
    if (key < node->key) node->left = insert(node->left, key);
    else if (key > node->key) node->right = insert(node->right, key);
    else return node;

    node->height = 1 + max(height(node->left), height(node->right));
    int balance = getBalance(node);

    if (balance > 1 && key < node->left->key) return rightRotate(node);
    if (balance < -1 && key > node->right->key) return leftRotate(node);
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {

```

```

        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

void preOrder(struct Node *root) {
    if(root != NULL) {
        printf("%d ", root->key);
        preOrder(root->left);
        preOrder(root->right);
    }
}

int main() {
    printf("Roll No: CH.SC.U4CSE24222\n");
    printf("Algorithm: AVL Tree Insertion\n-----\n");

    struct Node *root = NULL;
    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30); // This triggers a rotation

    printf("Preorder traversal of balanced AVL tree: ");
    preOrder(root);
    printf("\n");
    return 0;
}

```

OUTPUT:

```
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$ gcc avltree.c -o one
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$ ./one
Roll No: CH.SC.U4CSE24222
Algorithm: AVL Tree Insertion
-----
Preorder traversal of balanced AVL tree: 20 10 30
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$
```

2. Write a c program for Quick Sort

Code:

```
#include <stdio.h>
#include <stdlib.h>

enum Color { RED, BLACK };

struct Node {
    int data;
    enum Color color;
    struct Node *left, *right, *parent;
};

// Function to print roll number and headers
void printHeader() {
    printf("Roll No: CH.SC.U4CSE24222\n");
    printf("Algorithm: Red-Black Tree (Logic Overview)\n-----\n");
}

// Logic for Right Rotation
void rightRotate(struct Node **root, struct Node *y) {
    struct Node *x = y->left;
    y->left = x->right;
    if (x->right != NULL) x->right->parent = y;
    x->parent = y->parent;
    if (y->parent == NULL) *root = x;
    else if (y == y->parent->left) y->parent->left = x;
    else y->parent->right = x;
    x->right = y;
```

```

    y->parent = x;
}

// Logic for Left Rotation
void leftRotate(struct Node **root, struct Node *x) {
    struct Node *y = x->right;
    x->right = y->left;
    if (y->left != NULL) y->left->parent = x;
    y->parent = x->parent;
    if (x->parent == NULL) *root = y;
    else if (x == x->parent->left) x->parent->left = y;
    else x->parent->right = y;
    y->left = x;
    x->parent = y;
}

int main() {
    printHeader();
    printf("Roll No: CH.SC.U4CSE24222\n");
    printf("Red-Black Tree requires complex recoloring and rotation\n");
    return 0;
}

```

OUTPUT:

```
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$ gcc Red-Blacktree.c -o one
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$ ./one
Roll No: CH.SC.U4CSE24222
Algorithm: Red-Black Tree (Logic Overview)
-----
Roll No: CH.SC.U4CSE24222
Red-Black Tree requires complex recoloring and rotation logic.
kishore@kishore-LOQ-15IRX9: /media/kishore/Kishore Edu/Studies/sem 4/DAA/week 5$
```